

Algoritmi e Strutture Dati

Programmazione dinamica – Parte 3

Alberto Montresor

Università di Trento

2026/08/15

This work is licensed under a Creative Commons
Attribution-ShareAlike 4.0 International License.



Sommario

- 1 Prodotto di catena di matrici
- 2 Insieme indipendente di intervalli pesati
- 3 String matching approssimato

Ripasso - Moltiplicazione matrici

$$C = A \cdot B$$

$$c_{i,j} = \sum_{k=0}^{n_k-1} a_{i,k} \cdot b_{k,j}$$

$$\begin{aligned} T(n) &= \Theta(n_i \cdot n_k \cdot n_j) \\ &= \Theta(n^3) \end{aligned}$$

```
matrixProduct(real[][] A, B, C, int ni, nk, nj)
```

```
for i = 0 to ni - 1 do
```

% Righe

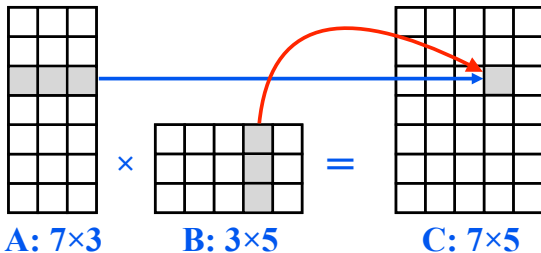
```
    for j = 0 to nj - 1 do
```

% Colonne

```
        C[i, j] = 0
```

```
        for k = 0 to nk - 1 do
```

```
            C[i, j] = C[i, j] + A[i, k] · B[k, j]
```



Prodotto di catena di matrici

Problema

Data una sequenza di n matrici $A_0, A_1, A_2, \dots, A_{n-1}$, compatibili due a due al prodotto, vogliamo calcolare il loro prodotto.

- Il prodotto di matrici non è **commutativo** ...
- ...ma è **associativo**: $(A \cdot B) \cdot C = A \cdot (B \cdot C)$

Cosa vogliamo ottimizzare

- Il prodotto di matrici si basa sulla **moltiplicazione scalare** come operazione elementare
- Vogliamo calcolare il prodotto delle n matrici impiegando **il più basso numero possibile di moltiplicazioni scalari**

Esempio 1 – Quanto costa moltiplicare queste 3 matrici?

A	100×1
B	1×100
C	100×1

	# Moltiplicazioni	Memoria
$(A \cdot B)$	$100 \times 1 \times 100 = 10000$	10000
$((A \cdot B) \cdot C)$	$\underline{100 \times 100 \times 1 = 10000}$	<u>100</u>
	20000	10100
$(B \cdot C)$	$1 \times 100 \times 1 = 100$	1
$(A \cdot (B \cdot C))$	$\underline{100 \times 1 \times 1 = 100}$	<u>100</u>
	200	101

Moltiplicare le matrici "in ordine" non è sempre una buona idea!

Esempio 2 – Quanto costa moltiplicare queste 4 matrici?

A	50×10
B	10×40
C	40×30
D	30×5

$(((A \cdot B) \cdot C) \cdot D) : 87500$ moltiplicazioni

$((A \cdot (B \cdot C)) \cdot D) : 34500$ moltiplicazioni

$((A \cdot B) \cdot (C \cdot D)) : 36000$ moltiplicazioni

$(A \cdot ((B \cdot C) \cdot D)) : 16000$ moltiplicazioni

$(A \cdot (B \cdot (C \cdot D))) : 10500$ moltiplicazioni

$(((A \cdot B) \cdot C) \cdot D) : 87500$

$(A \cdot B) \quad 50 \times 10 \times 40 = 20000$

$((A \cdot B) \cdot C) \quad 50 \times 40 \times 30 = 60000$

$((A \cdot B) \cdot C) \cdot D \quad \underline{50 \times 30 \times 5 = 7500}$

87500

Parentesizzazione

Come avete operato per risolvere il problema?

Come posso rappresentare una **parentesizzazione** di una catena di moltiplicazione di matrici?

Se avete utilizzato un approccio greedy (ingordo), dovete essere pronti a dimostrare la sua correttezza

Pensare "ricorsivamente" è fondamentale per applicare questa tecnica.

Parentesizzazione

Parentesizzazione

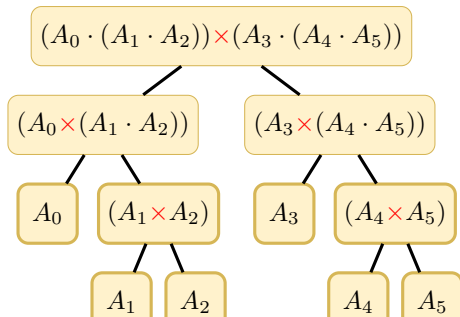
Una **parentesizzazione** $P_{i,j}$ del prodotto $A_i \cdot A_{i+1} \cdot \dots \cdot A_j$ consiste:

- nella matrice A_i , se $i = j$;
- nel prodotto di due parentesizzazioni $(P_{i,k} \cdot P_{k+1,j})$, altrimenti.

Esempio

$$(A_0 \cdot (A_1 \cdot A_2)) \times (A_3 \cdot (A_4 \cdot A_5))$$

In questo caso, $k = 2$ e il prodotto evidenziato è detto **"ultimo prodotto"**



Parentesizzazione ottima

Parentesizzazione ottima

La parentesizzazione che richiede il minor numero di moltiplicazioni scalari per essere completata, fra tutte le parentesizzazioni possibili.

Motivazione

Vale la pena preprocessare i dati per cercare la parentesizzazione migliore, per risparmiare tempo dopo nel calcolo vero e proprio

Domanda

Quante sono le parentesizzazioni possibili?

n	1	2	3	4	5	6	7	8	9	10
$NP[n]$	1	1	2	5	?	?	?	?	?	?

Parentesizzazione ottima

- $NP[n]$: numero di parentesizzazioni per n matrici
- L'ultimo prodotto può occorrere in... $n - 1$ posizioni diverse
- Sia $k \geq 1$ il numero di matrici a sinistra dell'ultimo prodotto
- A destra, ci sono $n - k$ matrici
 - $NP[k]$ parentesizzazioni per le k matrici a sinistra
 - $NP[n - k]$ parentesizzazioni per le $n - k$ matrici a destra

$$NP[n] = \begin{cases} 1 & n = 1 \\ \sum_{k=1}^{n-1} NP[k] \cdot NP[n - k] & n > 1 \end{cases}$$

n	1	2	3	4	5	6	7	8	9	10
$NP[n]$	1	1	2	5	14	42	132	429	1430	4862

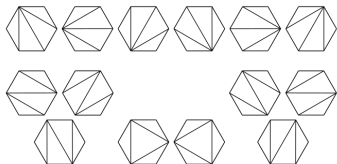
Parentesizzazione ottima

Numero di Catalan

$$NP[n] = C(n-1) = \frac{1}{n} \binom{2n-2}{n-1} = \Theta\left(\frac{4^n}{n\sqrt{n}}\right)$$

In matematica

$C(n)$: numero di modi in cui un poligono convesso con $n + 2$ lati può essere suddiviso in triangoli.



Esercizio

Dimostrare che $NP[n] = \Omega(2^n)$

Implicazione

Algoritmi di forza bruta non vanno quindi bene

Definizioni matematiche

$A_0 \cdot A_1 \cdot \dots \cdot A_{n-1}$	il prodotto di n matrici da ottimizzare
$c[i]$	il numero di righe della matrice A_i
$c[i + 1]$	il numero di colonne della matrice A_i
$A[i \dots j]$	il sottoprodotto $A_i \cdot A_{i+1} \cdot \dots \cdot A_j$
$P[i \dots j]$	una parentesizzazione per $A[i \dots j]$ (non necessariamente ottima)

Struttura di una parentesizzazione ottima

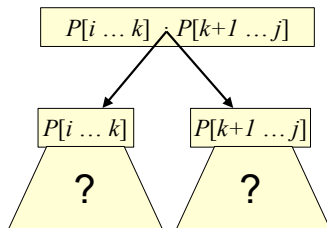
Osservazioni

- Sia $A[i \dots j]$ una sottosequenza del prodotto di matrici
- Si consideri una parentesizzazione ottima $P[i \dots j]$ di $A[i \dots j]$
- Esiste un **ultimo prodotto**: esiste un indice k tale che

$$P[i \dots j] = P[i \dots k] \cdot P[k+1 \dots j]$$

Domanda

Quali sono le caratteristiche dei due sottoprodotti $P[i \dots k]$ e $P[k+1 \dots j]$?



Teorema sottostruttura ottima

Teorema

Se $P[i \dots j] = P[i \dots k] \cdot P[k + 1 \dots j]$ è una parentesizzazione ottima del prodotto $A[i \dots j]$, allora:

- $P[i \dots k]$ è parentesizzazione ottima del prodotto $A[i \dots k]$
- $P[k + 1 \dots j]$ è parentesizzazione ottima del prodotto $A[k + 1 \dots j]$

Dimostrazione – per assurdo

- Supponiamo esista una parentesizzazione $P'[i \dots k]$ di $A[i \dots k]$ con costo inferiore a $P[i \dots k]$.
- Allora, $P'[i \dots k] \cdot P[k + 1 \dots j]$ sarebbe una parentesizzazione di $A[i \dots j]$ con costo inferiore a $P[i \dots j]$ che è ottima, il che è assurdo.

Valore della soluzione ottima

Definizione

Sia $DP[i][j]$ il minimo numero di moltiplicazioni scalari necessarie per calcolare il prodotto $A[i \dots j]$

- **Caso base:** $i = j$. Allora $DP[i][j] = 0$
- **Passo ricorsivo:** $i < j$. Esiste una parentesizzazione ottima

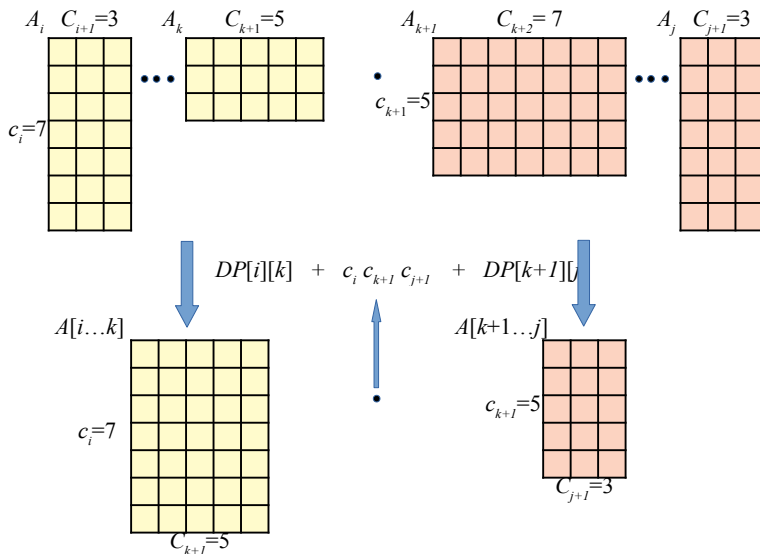
$$P[i \dots j] = P[i \dots k] \cdot P[k + 1 \dots j]$$

Sfruttando la ricorsione:

$$DP[i][j] = DP[i][k] + DP[k + 1][j] + c[i] \cdot c[k + 1] \cdot c[j + 1]$$

- $c[i] \cdot c[k + 1] \cdot c[j + 1]$ è il costo per moltiplicare
 - la matrice $A_i \dots A_k$: $c[i]$ righe, $c[k + 1]$ colonne
 - la matrice $A_{k+1} \dots A_j$: $c[k + 1]$ righe, $c[j + 1]$ colonne

Valore della soluzione ottima



Valore della soluzione ottima

Ma qual è il valore di k ?

- Non lo conosciamo....
- ... ma possiamo provarli tutti!
- k può assumere valori fra i e $j - 1$

Formula finale

$$DP[i][j] = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{ DP[i][k] + DP[k+1][j] + c[i]c[k+1]c[j+1] \} & i < j \end{cases}$$

Esempio

i/j	0	1	2	3	4	5
0	0					
1		0				
2			0			
3				0		
4					0	
5						0

$$\begin{aligned}
 DP[0][1] &= \min_{0 \leq k < 1} \{ DP[0][k] + DP[k+1][1] + c[0]c[k+1]c[2] \} \\
 &= DP[0][0] + DP[1][1] + c[0]c[1]c[2] \\
 &= c[0]c[1]c[2]
 \end{aligned}$$

Esempio

i/j	0	1	2	3	4	5
0	0					
1		0				
2			0			
3				0		
4					0	
5						0

$$\begin{aligned}
 DP[0][2] &= \min_{0 \leq k < 2} \{ DP[0][k] + DP[k+1][2] + c[0]c[k+1]c[3] \} \\
 &= \min \{ DP[0][0] + DP[1][2] + c[0]c[1]c[3], \\
 &\quad DP[0][1] + DP[2][2] + c[0]c[2]c[3] \}
 \end{aligned}$$

Esempio

i/j	0	1	2	3	4	5
0	0					
1		0				
2			0			
3				0		
4					0	
5						0

$$\begin{aligned}
 DP[1][4] &= \min_{1 \leq k < 4} \{ DP[1][k] + DP[k+1][4] + c[1]c[k+1]c[5] \} \\
 &= \min \{ DP[1][1] + DP[2][4] + c[1]c[2]c[5], \\
 &\quad DP[1][2] + DP[3][4] + c[1]c[3]c[5], \\
 &\quad DP[1][3] + DP[4][4] + c[1]c[4]c[5] \}
 \end{aligned}$$

Esempio

i/j	0	1	2	3	4	5
0	0					
1		0				
2			0			
3				0		
4					0	
5						0

$$\begin{aligned}
 DP[0][4] &= \min_{0 \leq k < 4} \{ DP[0][k] + DP[k+1][4] + c[0]c[k+1]c[5] \} \\
 &= \min \{ DP[0][0] + DP[1][4] + c[0]c[1]c[5], \\
 &\quad DP[0][1] + DP[2][4] + c[0]c[2]c[5], \\
 &\quad DP[0][2] + DP[3][4] + c[0]c[3]c[5], \\
 &\quad DP[0][3] + DP[4][4] + c[0]c[4]c[5] \}
 \end{aligned}$$

Esempio

i/j	0	1	2	3	4	5
0	0					
1		0				
2			0			
3				0		
4					0	
5						0

$$\begin{aligned}
 DP[0][5] &= \min_{0 \leq k < 5} \{ DP[0][k] + DP[k+1][5] + c[0]c[k+1]c[6] \} \\
 &= \min \{ \\
 &\quad DP[0][0] + DP[1][5] + c[0]c[1]c[6], \\
 &\quad DP[0][1] + DP[2][5] + c[0]c[2]c[6], \\
 &\quad DP[0][2] + DP[3][5] + c[0]c[3]c[6], \\
 &\quad DP[0][3] + DP[4][5] + c[0]c[4]c[6], \\
 &\quad DP[0][4] + DP[5][5] + c[0]c[5]c[6] \}
 \end{aligned}$$

Dalla formula al codice

Input

- Un vettore $c[0 \dots n]$ contenente le dimensioni delle matrici
 - $c[i]$ è il numero di righe della matrice A_i
 - $c[i + 1]$ è il numero di colonne della matrice A_i
- Due indici i, j che rappresentano l'intervallo di matrici da moltiplicare

Output

Il numero di moltiplicazioni scalari per calcolare il prodotto delle matrici comprese fra gli indici i e j

Approccio ricorsivo

```

int recPar(int[] c, int i, int j)


---


if i == j then
    | return 0
else
    | minSoFar =  $+\infty$ 
    | for int k = i to j - 1 do
    | | int
    | |   val = recPar(c, i, k) + recPar(c, k + 1, j) + c[i] · c[k + 1] · c[j + 1]
    | |   if val < minSoFar then
    | | |   minSoFar = val
    | return minSoFar

```

Complessità?

Valutazione

Alcune riflessioni

- La soluzione ricorsiva top-down è $\Omega(2^n)$
- Non è poi migliore dell'approccio basato su forza bruta!
- Il problema è che molti sottoproblemi vengono risolti più volte
- Il numero di sottoproblemi è $\frac{n(n+1)}{2}$

Versione bottom-up

Tabelle programmazione dinamica

Due matrici DP , $last$ di dimensione $n \times n$ tali che:

- $DP[i][j]$ contiene il numero di moltiplicazioni scalari necessarie per moltiplicare le matrici $A[i \dots j]$
- $last[i][j]$ contiene il valore k dell'ultimo prodotto che minimizza il costo per il sottoproblema

Versione bottom-up

```
computePar(int[] c, int n)
```

```
int[][] DP = new int[0...n-1][0...n-1]
```

```
int[][] last = new int[0...n-1][0...n-1]
```

```
for i = 0 to n - 1 do
```

```
    DP[i][i] = 0
```

% Fill main diagonal

```
for h = 1 to n - 1 do
```

% h: diagonal index

```
    for i = 0 to n - h - 1 do
```

% i: row

```
        int j = i + h
```

% j: column

```
        DP[i][j] = +∞
```

```
        for k = i to j - 1 do
```

% k: last product

```
            int temp = DP[i][k] + DP[k+1][j] + c[i] · c[k+1] · c[j+1]
```

```
            if temp < DP[i][j] then
```

```
                DP[i][j] = temp
```

```
                last[i][j] = k
```

```
return DP[0][n-1]
```

<i>DP</i>	0	1	2	3	4	5
0	0	224	176	218	276	350
1		0	64	112	174	250
2			0	24	70	138
3				0	30	90
4					0	90
5						0

<i>i</i>	<i>c[i]</i>
0	7
1	8
2	4
3	2
4	3
5	5
6	6

$$\begin{aligned}
 DP[0][3] &= \min_{0 \leq k < 3} \{ DP[0][k] + DP[k+1][3] + c[0] \cdot c[k+1] \cdot c[4] \} \\
 &= \min \{ DP[0][0] + DP[1][3] + c[0] \cdot c[1] \cdot c[4], \\
 &\quad \{ DP[0][1] + DP[2][3] + c[0] \cdot c[2] \cdot c[4], \\
 &\quad \{ DP[0][2] + DP[3][3] + c[0] \cdot c[3] \cdot c[4] \} \\
 &= \min \{ 0 + 112 + 7 \cdot 8 \cdot 3, \\
 &\quad \{ 224 + 24 + 7 \cdot 4 \cdot 3, \\
 &\quad \{ 176 + 0 + 7 \cdot 2 \cdot 3 \} \\
 &= \min \{ 280, 332, 218 \}
 \end{aligned}$$

<i>last</i>	0	1	2	3	4	5
0	0	0	0	2	2	2
1		0	1	2	2	2
2			0	2	2	2
3				0	3	4
4					0	4
5						0

<i>i</i>	<i>c[i]</i>
0	7
1	8
2	4
3	2
4	3
5	5
6	6

$$\begin{aligned}
 DP[0][3] &= \min_{0 \leq k < 3} \{ DP[0][k] + DP[k+1][3] + c[0] \cdot c[k+1] \cdot c[4] \} \\
 &= \min \{ DP[0][0] + DP[1][3] + c[0] \cdot c[1] \cdot c[4], \\
 &\quad \{ DP[0][1] + DP[2][3] + c[0] \cdot c[2] \cdot c[4], \\
 &\quad \{ DP[0][2] + DP[3][3] + c[0] \cdot c[3] \cdot c[4] \} \\
 &= \min \{ 0 + 112 + 7 \cdot 8 \cdot 3, \\
 &\quad \{ 224 + 24 + 7 \cdot 4 \cdot 3, \\
 &\quad \{ 176 + 0 + 7 \cdot 2 \cdot 3 \} \\
 &= \min \{ 280, 332, \mathbf{218} \}
 \end{aligned}$$

Parentesizzazione ottima

Considerazioni

- Il costo computazionale è $\mathcal{O}(n^3)$, in quanto ogni cella richiede tempo $\mathcal{O}(n)$ per essere riempita
- Il costo della funzione si trova nella posizione $DP[0][n - 1]$
- È anche necessario mostrare la soluzione trovata
- Per questo motivo abbiamo registrato informazioni sulla soluzione nella matrice *last*

Ricostruzione della soluzione – Stampa

```
computePar(int[] c, int n)
```

```
[...]
```

```
stampaPar(last, 0, n - 1)
```

```
stampaPar(int[][] last, int i, int j)
```

```
if i == j then
```

```
    | print "A["; print i; print "]"
```

```
else
```

```
    | print "("; stampaPar(last, i, last[i][j]); print ".";
    | stampaPar(last, last[i][j] + 1, j); print ")"
```

Ricostruzione della soluzione – Calcolo effettivo

```
int[][] multiply(matrix[] A, int[][] last, int i, int j)
```

```
if i == j then
```

```
    return A[i]
```

```
else
```

```
    int[][] X = multiply(A, last, i, last[i][j])
```

```
    int[][] Y = multiply(A, last, last[i][j] + 1, j)
```

```
    return matrix-multiplication(X, Y)
```

Esempio

$$A[0 \dots 5] = A[0 \dots 2] \cdot A[3 \dots 5]$$

$$A[0 \dots 2] = A[0] \cdot A[1 \dots 2]$$

$$A[3 \dots 5] = A[3 \dots 4] \cdot A[5]$$

$$A[1 \dots 2] = A[1] \cdot A[2]$$

$$A[3 \dots 4] = A[3] \cdot A[4]$$

<i>last</i>	0	1	2	3	4	5
0	0	0	0	2	2	2
1		0	1	2	2	2
2			0	2	2	2
3				0	3	4
4					0	4
5						0

Risultato finale

$$A = ((A_0 \cdot (A_1 \cdot A_2)) \cdot ((A_3 \cdot A_4) \cdot A_5))$$

Prodotto di catena di matrici

Take-home message – prendi e porta a casa

A volte, bisogna fare attenzione a come riempire la tabella - non è detto che riempire una riga dopo l'altra sia possibile

Algoritmi più efficienti

- Hu and Shing, 1982 – Complessità $\mathcal{O}(n \log n)$, basato su una visione geometrica del problema (triangolazione di poligoni)
- Wang, Zhu and Tian, 2013 – Complessità $\mathcal{O}(n \log m)$, dove m è il numero di "minimi locali" nel vettore delle dimensioni
- Nella pratica: questi due algoritmi vengono usati raramente per via dei costi moltiplicativi
- Nella pratica: strutturalmente simile al problema di ottimizzazione di join multipli in basi di dati

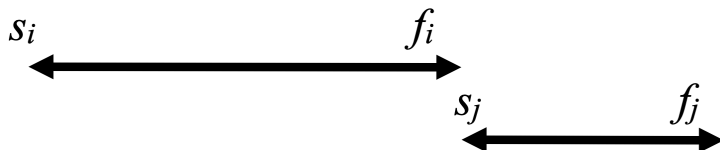
Insieme indipendente di intervalli pesati – Introduzione

Input

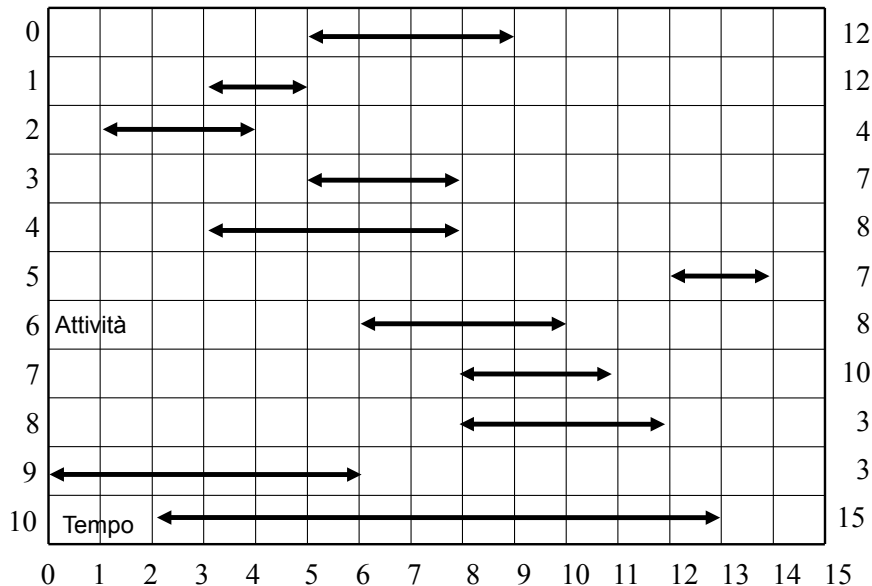
Siano dati n intervalli distinti $[s[0], f[0]), \dots, [s[n-1], f[n-1])$ della retta reale, aperti a destra, dove all'intervallo i è associato un profitto $w[i]$, $0 \leq i \leq n-1$.

Intervalli disgiunti

Due intervalli i e j si dicono **disgiunti** se: $f[j] \leq s[i]$ oppure $f[i] \leq s[j]$



Esempio



Definizione del problema

Problema

Trovare un **insieme indipendente di peso massimo**, ovvero un sottoinsieme $S \subseteq I = \{0, \dots, n-1\}$ di intervalli disgiunti tra loro tale che la somma dei loro profitti sia la più grande possibile.

Indipendenza

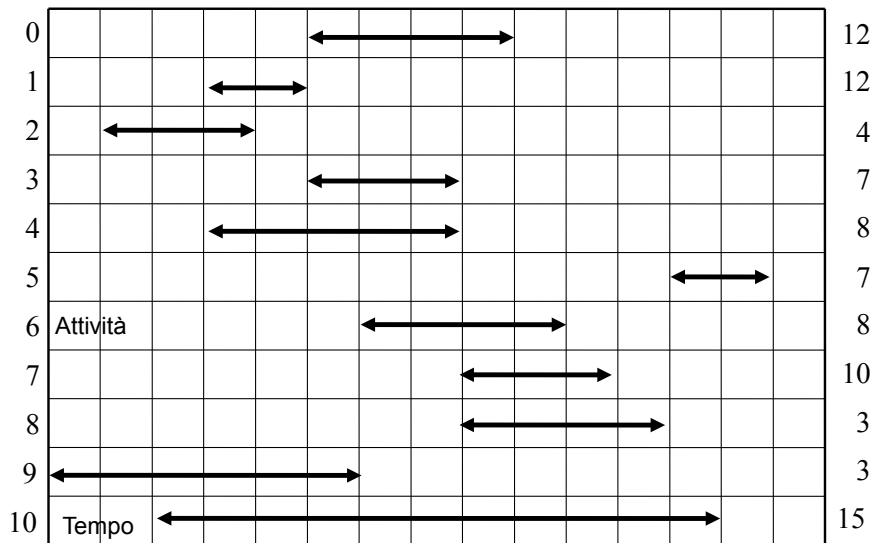
$$\forall i, j \in S, i \neq j \implies [s[i], f[i]) \cap [s[j], f[j]) = \emptyset$$

Ottimalità:

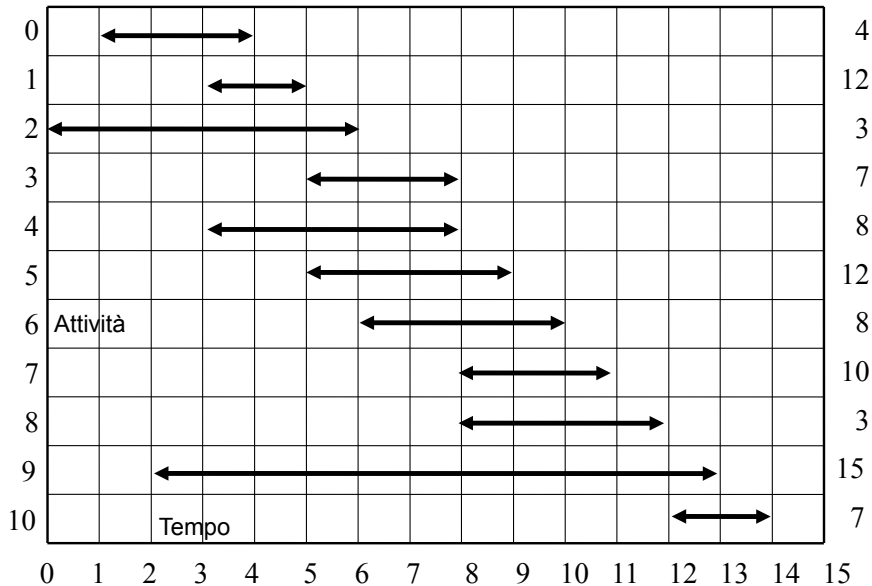
$$\max_{S \subseteq I} \sum_{i \in S} w[i]$$

Esempio: prenotazione di una sala conferenza

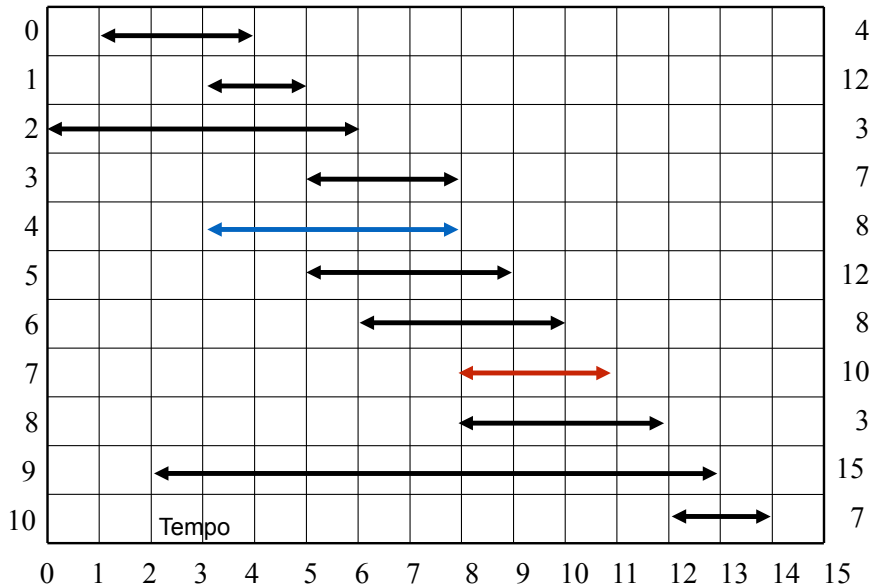
Come fareste? Pensate all'approccio prendo/non prendo



Esempio – Ordinato per tempi di fine non-decrescenti



Esempio – Cosa succede se prendo il rosso?



Pre-elaborazione

Per usare PD, è necessario effettuare una pre-elaborazione: ordinare gli intervalli per tempi di fine non decrescenti

$$f[0] \leq f[1] \leq \dots \leq f[n-1]$$

Profitto massimo (Versione 1)

$DP[i]$ contiene il profitto massimo ottenibile con i primi i intervalli

$$DP[i] = \begin{cases} 0 & i = 0 \\ \max \left(\begin{array}{l} DP[i-1], \quad \% \text{ non prendo} \\ w[i-1], \quad \% \text{ no intervalli compatibili} \\ \max\{DP[j] + w[i-1] : 0 < j < i \wedge f[j-1] \leq s[i-1]\} \end{array} \right) & i > 0 \end{cases}$$

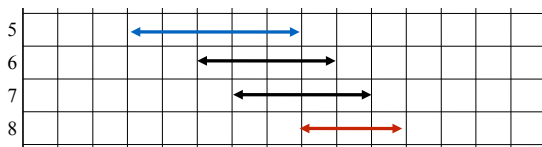
Costo computazionale?

$$\mathcal{O}(n^2)$$

Pre-elaborazione

Una seconda possibile pre-elaborazione consiste nel pre-calcolare il **predecessore** $pred[i] = j$ dell'intervallo $i - 1$, dove:

- $j < i$ è il massimo indice tale che $f[j - 1] \leq s[i - 1]$
- se non esiste tale indice, $pred[i] = 0$.



Profitto massimo (Versione 2)

$$DP[i] = \begin{cases} 0 & i = 0 \\ \max(DP[i - 1], DP[pred[i]] + w[i - 1]) & i > 0 \end{cases}$$

Pre-elaborazione - calcolo predecessori

```
int[] computePredecessor(int[] s, int[] f, int n)
```

```
int[] pred = new int[0...n]
```

```
pred[0] = 0
```

```
for i = 1 to n do
```

```
    j = i - 1
```

```
    while j > 0 and f[j - 1] > s[i - 1] do
```

```
        j = j - 1
```

```
    pred[i] = j
```

```
return pred
```

Quanto costa pre-calcolare i predecessori?

$\mathcal{O}(n^2)$

Si può fare meglio di così?

Sì! $\mathcal{O}(n \log n)$
(per esercizio)

Alcune note

- Gli intervalli vanno ordinati per tempo non decrescente di fine
- Eventuali intervalli con lo stesso tempo di fine possono essere ordinati in qualunque modo
- Questo perché ogni valore $DP[i]$ rappresenta il massimo profitto ottenibile con i primi i intervalli
- È quindi possibile escludere l'intervallo i -esimo se sceglierne uno precedente j (ma con lo stesso tempo di fine $f[i] = f[j]$) ha un valore $DP[j]$ più alto

Versione completa

```
SET maxinterval(int[ ] s, int[ ] f, int[ ] w, int n)
```

```
{ ordina gli intervalli per tempi di fine crescenti }
```

```
int[ ] pred = computePredecessor(s, f, n)
```

```
int[ ] DP = new int[0 . . . n]
```

```
DP[0] = 0
```

```
for i = 1 to n do
```

```
    DP[i] = max(DP[i - 1], w[i - 1] + DP[pred[i]])
```

```
i = n
```

```
SET S = Set()
```

```
while i > 0 do
```

```
    if DP[i - 1] > w[i - 1] + DP[pred[i]] then
```

```
        i = i - 1
```

```
    else
```

```
        S.insert(i - 1)
```

```
        i = pred[i]
```

```
return S
```

Costo computazionale

Costo computazionale

- Ordinamento intervalli: $\mathcal{O}(n \log n)$
- Calcolo predecessori: $\mathcal{O}(n \log n)$
- Riempimento tabella DP : $\mathcal{O}(n)$
- Ricostruzione soluzione: $\mathcal{O}(n)$
- Algoritmo completo: $\mathcal{O}(n \log n)$

Esercizio

Scrivere una funzione di calcolo predecessori in tempo $\mathcal{O}(n \log n)$

Insieme indipendente di intervalli pesati – Conclusioni

Take-home message – prendi e porta a casa

Talvolta, può essere necessario pre-processare l'input per poter applicare nella maniera più efficiente la programmazione dinamica

String matching approssimato

Definizione

Siano:

- $P = p_0 \dots p_{m-1}$ una stringa detta **pattern**
- $T = t_0 \dots t_{n-1}$ una stringa detta **testo**,

Un'**occorrenza k -approssimata** di P in T è una copia di P in T in cui sono ammessi k "errori" tra P e T , del seguente tipo:

- 1 i corrispondenti caratteri in P, T sono diversi (**sostituzione**)
- 2 un carattere in P non è incluso in T (**inserimento**)
- 3 un carattere in T non è incluso in P (**cancellazione**)

Esempio (Sostituzione, cancellazione)

$T =$ questo è un **o**scempio

$P =$ un **e**sempio

Esempio

Problema – Approximated string matching

Trovare un'occorrenza k -approssimata di P in T con k minimo ($0 \leq k \leq m$).

Spiegazione

- Qual è il minimo valore k per cui si trova un'occorrenza k -approssimata di P in T ?
- Dove si trova?
- Con quali errori?

Sottostruttura ottima

Definizione

Sia $DP[0 \dots m][0 \dots n]$ una tabella di programmazione dinamica tale che $DP[i][j]$ sia il minimo valore k per cui esiste un'occorrenza k -approssimata di $P(i)$ in $T(j)$ che termina nel carattere $T[j - 1]$

Quattro possibilità

$DP[i - 1][j - 1]$, se $P[i - 1] = T[j - 1]$

$DP[i - 1][j - 1] + 1$, se $P[i - 1] \neq T[j - 1]$

$DP[i - 1][j] + 1$

$DP[i][j - 1] + 1$

avanza su entrambi i caratteri (uguaglianza)

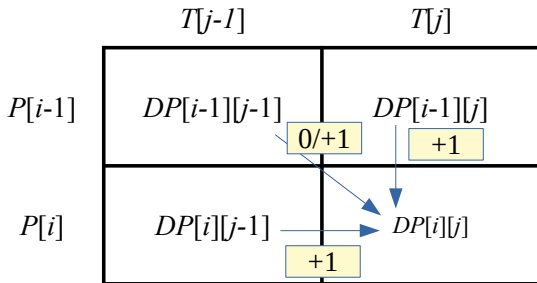
avanza su entrambi i caratteri (sostituzione)

avanza sul pattern (inserimento)

avanza sul testo (cancellazione)

Sottostruttura ottima

$$DP[i][j] = \begin{cases} 0 & i = 0 \\ i & j = 0 \\ \min \{ DP[i-1][j-1] + \delta, & \delta = \text{iif}(P[i-1] == T[j-1], 0, 1) \\ DP[i-1][j] + 1, \\ DP[i][j-1] + 1 \} & \text{altrimenti} \end{cases}$$



Dove si trova la soluzione finale?

- $DP[m][j] = k$ se e solo se esiste un'occorrenza k -approssimata di P in $T(j)$ che termina nella posizione $j - 1$.
- La soluzione del problema è data dal più piccolo valore $DP[m][j]$, per $1 \leq j \leq n$

		T					
P		A B A B A					
		0	0	0	0	0	0
B	A	1	1	0	1	0	1
	B	2	1	1	0	1	0
	A	3	2	1	1	0	1
	B						

Algoritmo

```

int stringMatching(item[] P, item[] T, int m, int n)


---


int[][] DP = new int[0...m][0...n]
for j = 0 to n do DP[0][j] = 0                                % Caso base: i = 0
for i = 1 to m do DP[i][0] = i                                % Caso base: j = 0
for i = 1 to m do                                              % Caso generale
    for j = 1 to n do
        %  $P_i \leftrightarrow P[i-1], t_j \leftrightarrow T[j-1]$ 
        DP[i][j] = min(DP[i - 1][j - 1] + iif(P[i - 1] == T[j - 1], 0, 1),
            DP[i - 1][j] + 1,
            DP[i][j - 1] + 1)
    end for
int pos = 1                                                    % Calcola minimo ultima riga
for j = 2 to n do
    if DP[m][j] < DP[m][pos] then
        pos = j
    end if
end for
return pos - 1

```

String matching approssimato

Take-home message – prendi e porta a casa

- Non è detto che la "soluzione finale" si trovi nella casella "in basso a destra";
- È invece possibile che la soluzione debba essere ricercata essa stessa nella tabella *DP*

Reality check

Approximate String Matching è un esempio di **string metric**:

[...] is a metric that measures distance ("inverse similarity") between two strings [...]

String metrics are used heavily in information integration and are currently used in areas including fraud detection, fingerprint analysis, plagiarism detection, ontology merging, DNA analysis, RNA analysis, image analysis, evidence-based machine learning, database data deduplication, data mining, incremental search, data integration, and semantic knowledge integration.

https://en.wikipedia.org/wiki/String_metric

Esempi

Edit distance, detta anche **distanza di Levenshtein**

Per concludere

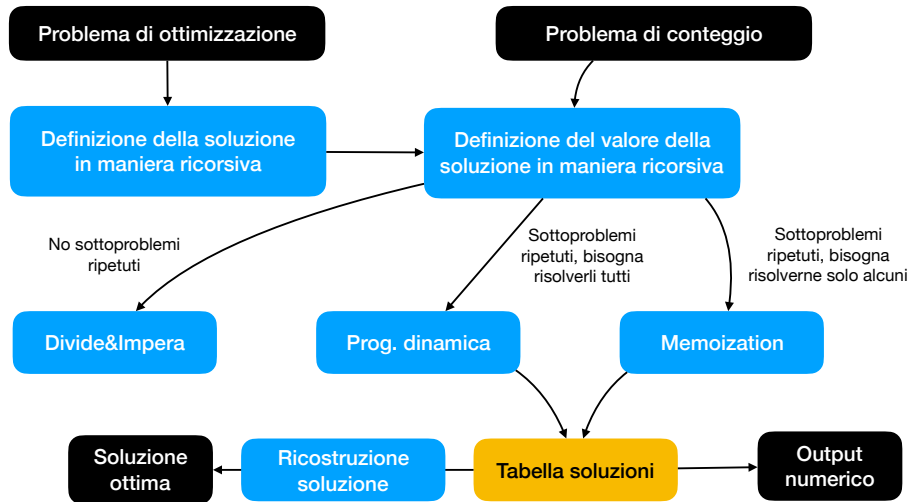
Una lezione ancora più importante

La programmazione dinamica non è la soluzione di tutti i vostri problemi. Esistono altre tecniche che possono fare "meglio di così". Inoltre, è possibile che soluzioni ad-hoc possano essere migliori

Esempi

- **Moltiplicazione di matrici:** esiste soluzione $\mathcal{O}(n \log n)$
- **Longest common subsequence:** può essere risolto in tempo $\mathcal{O}(mn / \log n)$ (con alfabeto limitato)
- **Four Russians algorithm:** tecnica generale che può essere applicata a vari problemi su matrice con alfabeto limitato $\mathcal{O}(n^2 / \log n)$
 - Edit distance
 - Transitive closure, ...

Approccio generale



Riassunto: programmazione dinamica / memoization

Fasi

- Caratterizzare la **struttura** di una soluzione ottima
- Dimostrare che la soluzione gode di **sottostruttura ottima**
- Definire ricorsivamente il **valore** di una soluzione ottima
- Calcolare il **valore** di una soluzione ottima "bottom-up" (prog. dinamica) / "top-down" (memoization)
- **Ricostruzione** di una soluzione ottima